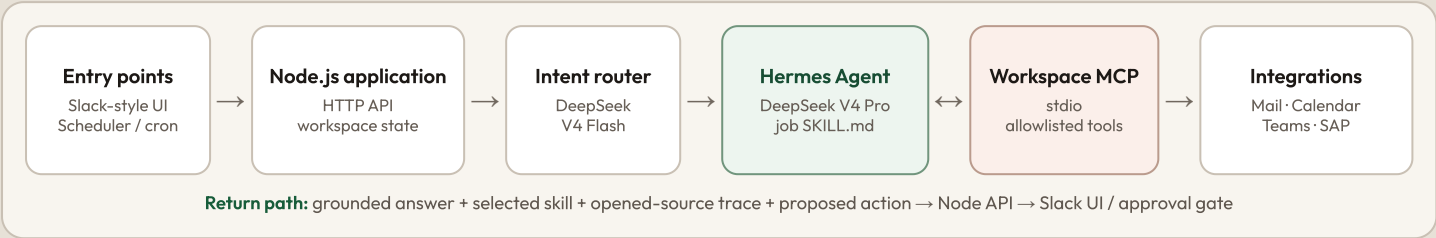


John — Technical Solution Architecture

Objective: give one scoped AI agent the context, skills, and tools required to produce a grounded next action—without unrestricted system access.

Working prototype · Hermes + MCP

SYSTEM ARCHITECTURE



TECHNICAL STACK

Frontend
Semantic HTML, CSS, vanilla JavaScript; Slack interaction, integration setup, source inspection, workflow visualisation, and trace.

Application
Node.js 22, framework-free HTTP server, static assets and JSON APIs, one process on port 4740.

Models
OpenRouter gateway. V4 Flash routes; V4 Pro performs bounded, tool-using reasoning.

Agent
Hermes one-shot runtime; versioned SKILL.md per job; app-specific HERMES_HOME.

Tools + data
MCP over stdio; read_source and gated draft storage; JSON integration records and action state.

Testing + deploy
Playwright smoke test and npm scripts; deploy behind Caddy/nginx or a secure tunnel.

REQUEST EXECUTION

- server.mjs receives a Slack request or scheduled event and validates workspace readiness.
- Flash returns an allowed route label; cross-workspace requests stop before Hermes.
- The server starts Hermes with one skill, provider, worker model, and bounded turn budget.
- Hermes calls MCP; MCP checks the source allowlist and connection state.
- The server returns answer, skill, source trace, and proposed action.

SECURITY AND CONTROL PLANE

- Least privilege:** source IDs, never arbitrary paths.
- Enforced below prompts:** server + MCP own access decisions.
- Revocation:** removing a source blocks the next call immediately.
- Tool restriction:** terminal, web, browser, general files, and sending disabled.
- Action separation:** read, propose, approve, and execute are distinct.
- Audit:** visible skill/tool trace; secrets remain server-side.

OPERATIONAL TOPOLOGY AND FAILURE HANDLING

Process boundary
The browser is untrusted and receives no secrets. Node owns workspace state and starts bounded Hermes child processes; Hermes launches MCP over stdio.

API + observability
/api/workspace, /api/source/:id, /api/ask, and /api/health expose setup, inspection, execution, and readiness. Session events produce the visible trace.

Failure posture
Refusals stop before tools; missing sources disable Ask; tool/model errors surface explicitly. Production adds queues, retry/idempotency, metrics, alerts, and encrypted connector credentials.

KEY ARCHITECTURE DECISIONS

Two-model split
Fast, inexpensive routing; stronger reasoning only for grounded multi-source work.

Skills, not one prompt
Each job has versioned instructions, a bounded turn budget, and a reviewable tool surface.

MCP integration seam
Connector credentials stay outside the model; adapters can change without rewriting skills.

Deterministic control
Code owns access, refusal, approvals, and audit; the model owns interpretation and composition.

PROTOTYPE IMPLEMENTATION VS PRODUCTION ADAPTERS

IMPLEMENTED AND EXECUTABLE
Node API/UI, live inference, Hermes skills, MCP isolation, inspectable payloads, source revocation, refusal, and trace.

PRODUCTION SUBSTITUTION
Replace local adapters with Microsoft Graph, Teams/meeting APIs, SAP events/APIs, Slack APIs, durable storage, and Hermes cron or an enterprise scheduler. The skill/MCP boundary remains unchanged.

Models reason over context; deterministic code controls access, actions, and audit.